Machine Learning Mini-Project 4

Regularization, Optimization, CNNs, LSTMs, and Word Embeddings

A Practical Study on Synthetic Regression, MNIST, CIFAR-10, IMDB, and a Custom Text Corpus

Course: Principles of Machine Learning
Instructor: Dr. Emami

Name: *Hossein Mirsaeidi*
Student ID: *403211408*

Summer 2026

Abstract

This project studies several core ideas of modern deep learning through five connected tasks. First, I write the forward and backward equations of a small feedforward network and carry out one update step by hand. Second, I train a multilayer perceptron on a noisy cubic function and compare early stopping with dropout. Third, I train an MLP on MNIST and contrast the SGD and Adam optimizers. Fourth, I build a convolutional network for CIFAR-10 image classification and a long short-term memory network for IMDB sentiment analysis, and I compare their behaviour. Fifth, I implement a Continuous Bag-of-Words model and visualise the learned word embeddings, both directly in two dimensions and in ten dimensions reduced by PCA. Throughout, the focus is on careful evaluation and a clear discussion of why each method behaves the way it does. All experiments use TensorFlow/Keras and were run on a 4 GB GPU.

Contents

1 Deep Feedforward Networks and Backpropagation

I consider a fully connected network with the architecture

$$\text{Input (2)} \;\to\; \text{Hidden (3, ReLU)} \;\to\; \text{Hidden (2, ReLU)} \;\to\; \text{Output (1, linear)},$$

with scalar target y and input $\mathbf{x} = [x_1, x_2]^\top$.

1.1 Forward-pass equations

Let the parameters be $W^{[1]} \in \mathbb{R}^{3\times 2},\ \mathbf{b}^{[1]} \in \mathbb{R}^3$, $W^{[2]} \in \mathbb{R}^{2\times 3},\ \mathbf{b}^{[2]} \in \mathbb{R}^2$, and $W^{[3]} \in \mathbb{R}^{1\times 2},\ b^{[3]} \in \mathbb{R}$. The forward pass is

$$\mathbf{z}^{[1]} = W^{[1]}\mathbf{x} + \mathbf{b}^{[1]}, \qquad \mathbf{a}^{[1]} = \mathrm{ReLU}\big(\mathbf{z}^{[1]}\big), \tag{1}$$

$$\mathbf{z}^{[2]} = W^{[2]}\mathbf{a}^{[1]} + \mathbf{b}^{[2]}, \qquad \mathbf{a}^{[2]} = \mathrm{ReLU}\big(\mathbf{z}^{[2]}\big), \tag{2}$$

$$z^{[3]} = W^{[3]}\mathbf{a}^{[2]} + b^{[3]}, \qquad \hat{y} = z^{[3]}, \tag{3}$$

where $\mathrm{ReLU}(z) = \max(0, z)$ acts element-wise and the output neuron is linear.

1.2 Backpropagation for the first layer

With the squared loss $\mathcal{L} = \frac{1}{2}(y-\hat{y})^2$ I define the local gradients (deltas) and push them backwards through the network. Because the output is linear,

$$\delta^{[3]} = \frac{\partial \mathcal{L}}{\partial z^{[3]}} = -(y - \hat{y}), \tag{4}$$

$$\boldsymbol{\delta}^{[2]} = \big(W^{[3]\top}\delta^{[3]}\big) \odot \mathrm{ReLU}'\big(\mathbf{z}^{[2]}\big), \tag{5}$$

$$\boldsymbol{\delta}^{[1]} = \big(W^{[2]\top}\boldsymbol{\delta}^{[2]}\big) \odot \mathrm{ReLU}'\big(\mathbf{z}^{[1]}\big), \tag{6}$$

where $\mathrm{ReLU}'(z) = \mathbf{1}[z > 0]$ and $\odot$ is the element-wise product. The gradient with respect to the first-layer parameters and the corresponding gradient-descent update with learning rate η are

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \boldsymbol{\delta}^{[1]}\mathbf{x}^\top, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{[1]}} = \boldsymbol{\delta}^{[1]}, \qquad W^{[1]} \leftarrow W^{[1]} - \eta\,\boldsymbol{\delta}^{[1]}\mathbf{x}^\top. \tag{7}$$

1.3 A single numerical update

I use $\mathbf{x} = [1, -1]^\top$, $y = 2$, $\eta = 0.1$, and

$$W^{[1]} = \begin{bmatrix} 0.1 & -0.2 \\ 0.05 & 0.3 \\ -0.3 & 0.1 \end{bmatrix}, \quad \mathbf{b}^{[1]} = \mathbf{0}, \quad W^{[2]} = \begin{bmatrix} 0.1 & 0.2 & -0.4 \\ 0.01 & -0.3 & 0.2 \end{bmatrix}, \quad \mathbf{b}^{[2]} = \mathbf{0},$$

$$W^{[3]} = \begin{bmatrix} 0.4 & -0.1 \end{bmatrix}, \qquad b^{[3]} = 0.$$

Remark on the data. The assignment lists $W^{[3]} = [0.4, -0.1, 0.2]$ with three weights, but the second hidden layer has only two neurons (and $\mathbf{b}^{[2]} \in \mathbb{R}^2$). To keep the network consistent with the stated $2 \to 3 \to 2 \to 1$ shape I use $W^{[3]} = [0.4, -0.1]$ and treat the extra value as a typo. A NumPy computation and a TensorFlow automatic-differentiation check in the notebook both confirm the numbers below.

Forward pass.

$$\mathbf{z}^{[1]} = [0.3,\ -0.25,\ -0.4]^\top, \qquad \mathbf{a}^{[1]} = [0.3,\ 0,\ 0]^\top,$$

$$\mathbf{z}^{[2]} = [0.03,\ 0.003]^\top, \qquad \mathbf{a}^{[2]} = [0.03,\ 0.003]^\top,$$

$$\hat{y} = 0.0117, \qquad \mathcal{L} = \tfrac{1}{2}(2 - 0.0117)^2 = 1.9767.$$

Backward pass. With $\delta^{[3]} = -(2 - 0.0117) = -1.9883$,

$$\boldsymbol{\delta}^{[2]} = [-0.79532,\ 0.19883]^\top, \qquad \boldsymbol{\delta}^{[1]} = [-0.07754,\ 0,\ 0]^\top,$$

because only the first pre-activation of each hidden layer is positive. The first-layer gradient and its update are

$$\frac{\partial \mathcal{L}}{\partial W^{[1]}} = \boldsymbol{\delta}^{[1]}\mathbf{x}^\top = \begin{bmatrix} -0.07754 & 0.07754 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \qquad W^{[1]}_{\text{new}} = \begin{bmatrix} 0.10775 & -0.20775 \\ 0.05 & 0.3 \\ -0.3 & 0.1 \end{bmatrix}.$$

For completeness, the other updated parameters are

$$W^{[3]}_{\text{new}} = [0.40596,\ -0.09940], \quad b^{[3]}_{\text{new}} = 0.19883, \quad \mathbf{b}^{[2]}_{\text{new}} = [0.07953,\ -0.01988]^\top,$$

$$W^{[2]}_{\text{new}} = \begin{bmatrix} 0.12386 & 0.2 & -0.4 \\ 0.00404 & -0.3 & 0.2 \end{bmatrix}, \qquad \mathbf{b}^{[1]}_{\text{new}} = [0.00775,\ 0,\ 0]^\top.$$

Only weights connected to active (non-zero) ReLU units receive an update, which is a direct consequence of the ReLU$'$ gate in the backward pass.

2 Regularization for Deep Learning

2.1 Effect of L2 regularization

L2 regularization adds a penalty on the squared norm of the weights to the loss,

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2}\sum_i w_i^2. \tag{8}$$

The gradient of each weight gains an extra λw term, so the update becomes

$$w \leftarrow w - \eta\left(\frac{\partial \mathcal{L}}{\partial w} + \lambda w\right) = (1 - \eta\lambda)\, w - \eta\frac{\partial \mathcal{L}}{\partial w}. \tag{9}$$

Every step first shrinks the weight by the factor $(1 - \eta\lambda)$ and then applies the ordinary gradient step. This is why L2 is also called weight decay: it constantly pulls the weights toward zero, which favours smaller and smoother models and reduces variance.

2.2 Dropout during training and inference

During training, dropout keeps each unit of a layer with probability p and zeroes the rest using a fresh random mask for every mini-batch. With inverted dropout the surviving activations are scaled by $1/p$ so that the expected output of the layer stays the same. This stops units from co-adapting and is equivalent to training an ensemble of many thinned sub-networks that share weights. During inference, dropout is turned off: all units are active and no scaling is applied, because the $1/p$ factor was already absorbed in training. The prediction then behaves like an average over the sub-networks. In Keras the argument of `Dropout` is the drop probability, so `Dropout(0.5)` removes half of the units.

2.3 Early stopping vs. dropout vs. L2 on small data

Early stopping watches the validation loss and halts once it stops improving, restoring the best weights; it controls overfitting by limiting how long the optimizer runs, costs almost nothing, and only needs a validation split. Dropout injects multiplicative noise and helps most for large over-parameterised networks, but on a small network dropping half of the units can remove too much capacity and cause underfitting, and it needs the full training budget. L2 shrinks weights smoothly and is stable, but its strength λ must be tuned. On a small dataset early stopping is usually the most convenient and best-performing choice, dropout tends to over-regularise tiny models, and L2 lies in between.

2.4 Experiment: MLP regressor on a noisy cubic

I draw $N = 1000$ points uniformly from $[-10, 10]^2$, evaluate $f(x_1, x_2) = x_1^3 - x_1^2 + 2x_2^2 + 3x_1x_2 + 5$, add noise $\varepsilon \sim \mathcal{U}(-1, 1)$, and split 80/20. Because the target spans roughly $\pm 10^3$, I standardize both inputs and target using training-set statistics and report the test MSE back in the original units. The model is an MLP with two hidden layers of ten ReLU units and a linear output, trained with Adam and MSE for up to 200 epochs with a 10% validation split.

Figure 1 shows the baseline learning curves and the true-versus-predicted scatter on the test set. The baseline reaches a test MSE of about 820 (in the original units) after roughly 25 s of training; the scatter plot lies close to the diagonal, so the network recovers the cubic surface well.

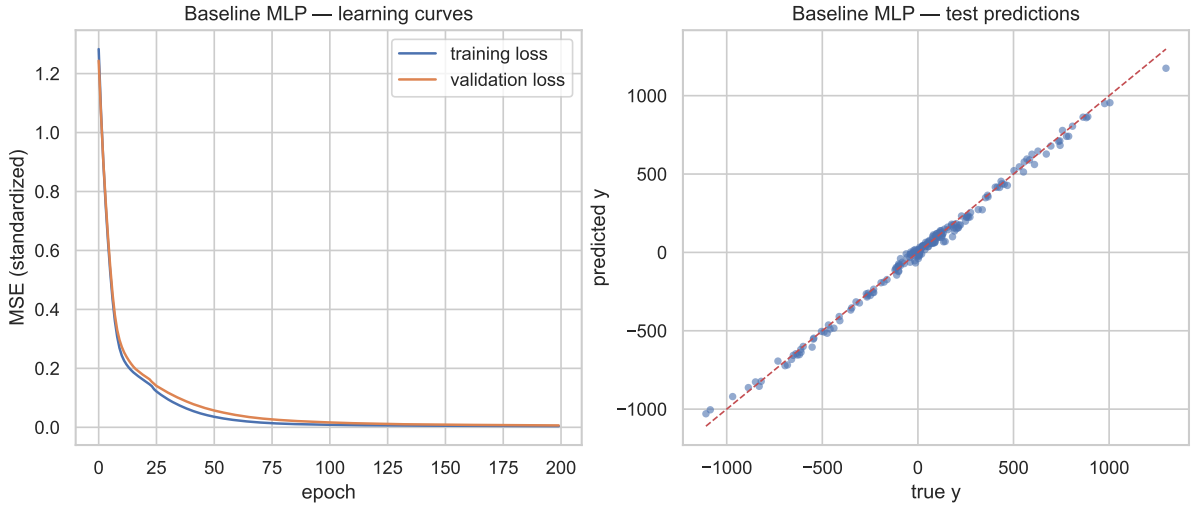


Figure 1: Baseline MLP: training/validation loss (left) and test predictions vs. true values (right).

I then retrain the same network with (a) early stopping (patience 10, best weights restored) and (b) dropout $p = 0.5$ after each hidden layer for the full 200 epochs. Table 1 and Figure 2 summarise the comparison.

Table 1: Regularization strategies on the regression task.

Strategy	Epochs	Training time (s)	Test MSE (original units)
Baseline (200 epochs)	200	25.4	820.4
Early stopping (patience 10)	200 (never triggered)	24.6	820.4
Dropout ($p = 0.5$, 200 epochs)	200	24.0	27,191.9

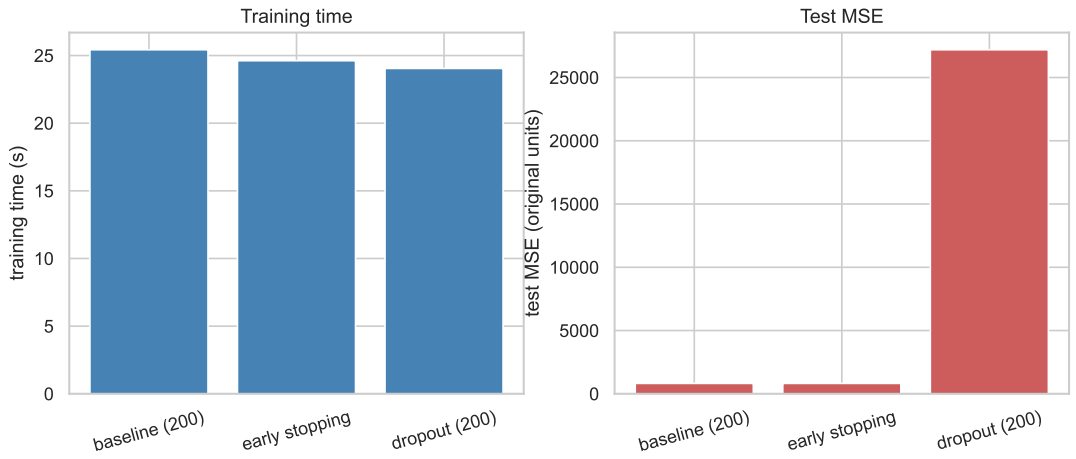


Figure 2: Training time and test MSE for the three training strategies.

Recommendation. The added noise here ($\varepsilon \in [-1, 1]$) is tiny compared to the signal, which reaches about $\pm 10^3$, so the model does not overfit within 200 epochs: the validation loss keeps decreasing and *early stopping never triggers*. Its epoch count, training time and test MSE therefore match the baseline exactly. Dropout at $p = 0.5$, by contrast, is far too aggressive for a network with only ten units per layer; it removes so much capacity that the test MSE jumps from about 820 to about 27,000. For this low-noise regression I therefore recommend **early stopping** as a safe, almost cost-free safeguard – it simply does nothing when there is no overfitting to catch – whereas heavy dropout is clearly harmful.

3 Optimization for Training Deep Networks

3.1 Update rules: SGD, Momentum, Adam

Let $g_t = \nabla_\theta \mathcal{L}(\theta_t)$ be the gradient at step t.

- **SGD:** $\theta_{t+1} = \theta_t - \eta\, g_t$.
- **Momentum:** $v_t = \mu v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t$, where μ (e.g. 0.9) builds a velocity that damps oscillations and speeds progress along consistent directions.
- **Adam:** bias-corrected first and second moments,

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \theta_{t+1} = \theta_t - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

  Adam adapts a per-parameter step size, combining momentum with an RMS normalisation.

3.2 Vanishing gradients

In a deep network the gradient of an early layer is a product of many Jacobians. With saturating activations such as sigmoid or tanh, each factor has a derivative below one, so the product shrinks exponentially with depth and the early layers barely learn; poor weight initialisation makes this worse. Two effective mitigations are: (i) use non-saturating activations (ReLU) together with variance-preserving initialisation (He/Xavier); and (ii) add batch normalisation and/or residual (skip) connections, which keep activation statistics stable and give the gradient a shorter path back to the early layers.

3.3 Experiment: SGD vs. Adam on MNIST

I load MNIST, scale the pixels to $[0, 1]$, one-hot encode the labels, and train two identical $784 \to 128 \to 64 \to 10$ MLPs for 20 epochs: one with SGD (learning rate 0.01, momentum 0.9) and one with Adam (defaults). Figure 3 overlays the learning curves. The final test accuracies are 97.68% for SGD and 97.51% for Adam (training took 23.2 s and 18.6 s respectively).

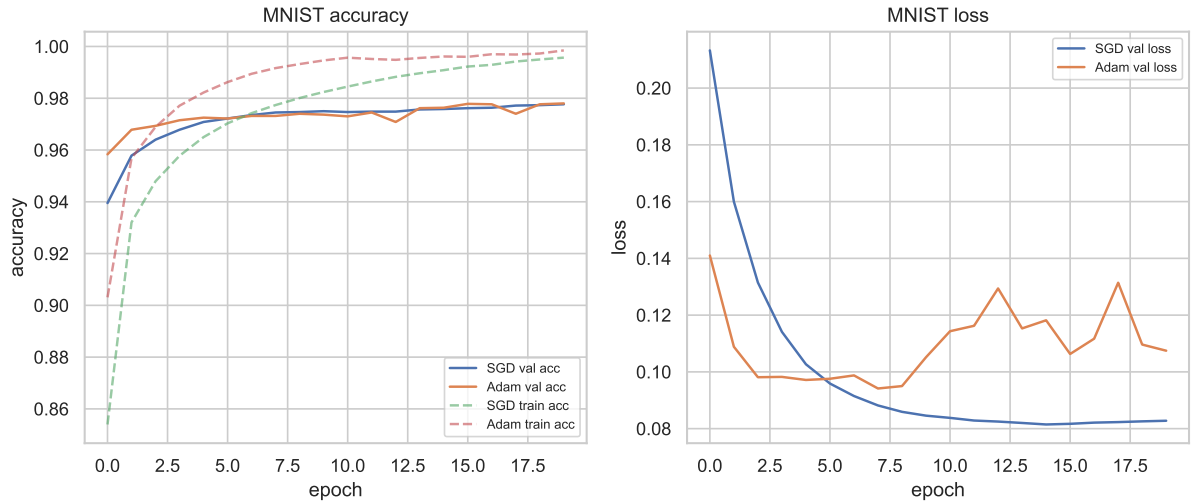


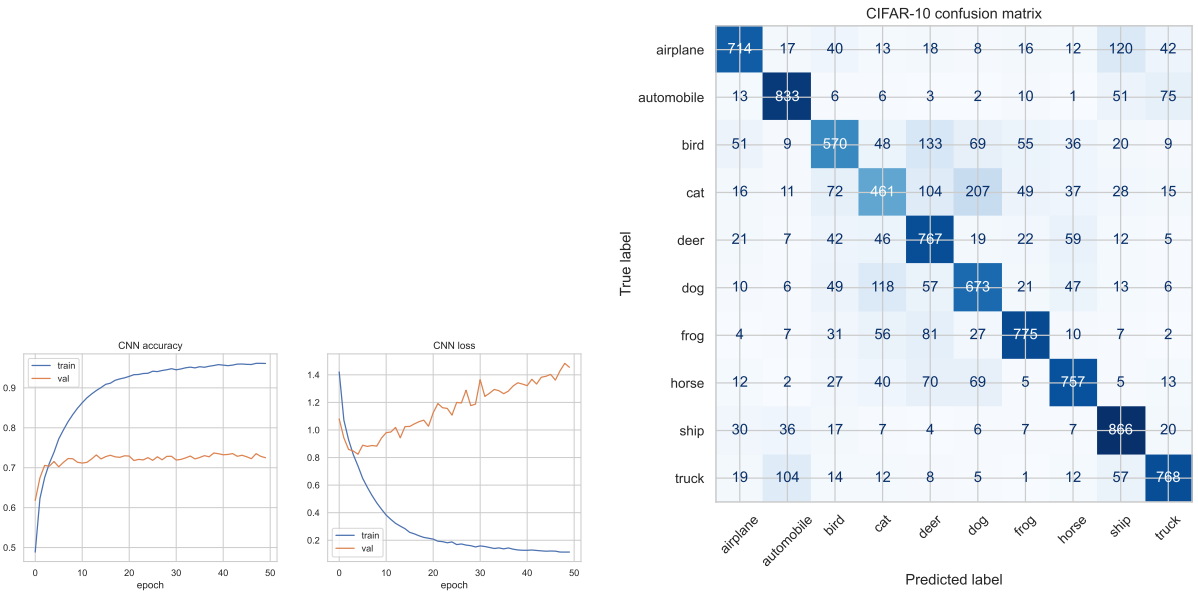
Figure 3: MNIST: accuracy (left) and loss (right) per epoch for SGD and Adam.

Comment. Adam reaches a low loss within the first few epochs, while SGD climbs more gradually (Figure 3). On this easy task the two finish very close (97.68% vs. 97.51%), and here SGD with momentum even ends a little higher. This matches the usual trade-off: Adam converges faster, but a well-tuned SGD with momentum can match or slightly beat its final accuracy.

4 CNN vs. LSTM on Benchmark Datasets

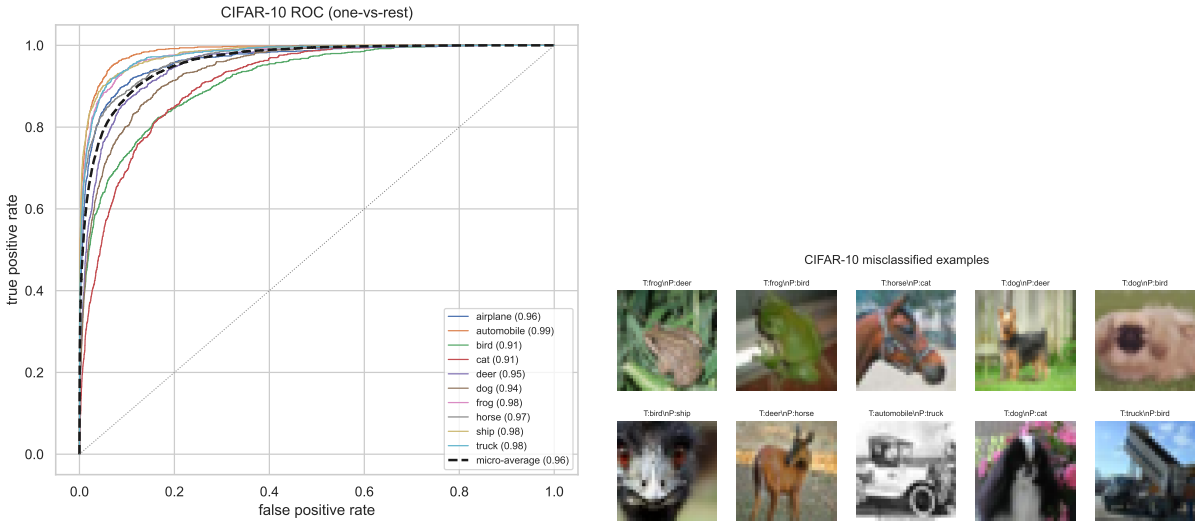
4.1 CNN on CIFAR-10

I load CIFAR-10 (50,000 training and 10,000 test images), scale the pixels to $[0, 1]$, and one-hot encode the ten classes. The network has one convolutional block (Conv2D 32, Conv2D 64, MaxPooling, Dropout 0.25), then Flatten $\to$ Dense(512) $\to$ Dropout(0.5) $\to$ Dense(10, softmax). It is trained with Adam and categorical cross-entropy for 50 epochs (batch 64, 10% validation). The final test accuracy is 71.8% and training took about 6.7 min, with a peak GPU memory of only 0.89 GB.



(a) Learning curves. **(b)** Confusion matrix.

Figure 4: CIFAR-10 CNN: training behaviour and per-class confusion.



(a) One-vs-rest ROC curves. **(b)** A few misclassified images.

Figure 5: CIFAR-10 CNN: ROC analysis and example errors.

Observations. The errors concentrate among visually similar classes: the most frequent mistakes are cat read as dog (207 images), bird as deer (133), and airplane as ship (120). The ROC curves agree – the animal classes have the lowest one-vs-rest AUC (cat and bird about 0.91) while rigid man-made objects score highest (automobile, ship, and truck about 0.98), and the micro-average AUC is 0.960. The learning curves make the overfitting plain: training accuracy climbs above 0.95 while validation accuracy stalls near 0.73 and the validation loss rises steadily after a few epochs, which is why the test accuracy settles at a moderate 71.8%.

4.2 LSTM on IMDB sentiment

Each review is an integer sequence over the 20,000 most frequent words, with rarer words mapped to `<UNK>`; sequences are padded/truncated to 200 tokens (zeros prepended, extra words at the

end dropped). The model is Embedding(20000,128) → LSTM(128) → Dense(64, ReLU) → Dropout(0.5) → Dense(1, sigmoid), trained with Adam and binary cross-entropy for 20 epochs. On the test set I obtain accuracy 81.2%, precision 0.824, recall 0.794, and F1 = 0.809 (training took about 2.4 min).

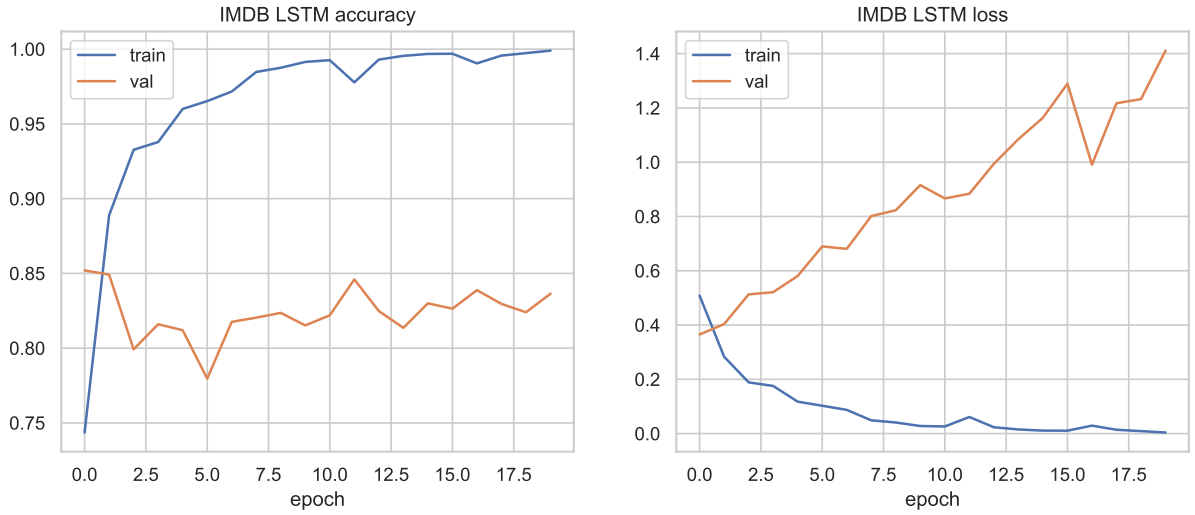


Figure 6: IMDB LSTM: accuracy (left) and loss (right) per epoch.

Observations. The model overfits quickly: after only one or two epochs the training accuracy approaches 100% while the validation loss begins to climb (from about 0.37 to well above 1.0), even though validation accuracy stays around 0.82–0.85. The mistakes are mostly reviews with mixed or ironic sentiment – a largely negative review that ends on a warm note, or sarcasm and praise-by-comparison – where a single averaged sequence state struggles to weigh the decisive words. Stopping earlier or regularizing more strongly would likely generalise a little better.

5 Continuous Bag-of-Words Word Embeddings

I train a CBOW model on a roughly 1000-word text about machine learning and neural networks (from Wikipedia, stored in `data/cbow_corpus.txt`). With a context window of two words on each side, the model averages the context embeddings and predicts the centre word through a softmax over the vocabulary, trained with Adam and cross-entropy.

5.1 Direct 2-D embedding

I first learn embeddings of dimension $d = 2$ for at least 300 epochs and plot them directly (Figure 7); all 454 words are plotted as points, and for readability only the most frequent ones carry a text label. With only 454 distinct words and about a thousand training pairs, the two-dimensional space is very tight and the neighbourhoods are noisy. Some domain words still drift together – the nearest words to *learning* include *deep* and *loss*, and those of *network* include *algorithm* – but many associations are spurious, and rare words scatter around the edges. The clusters are therefore only loosely formed, which is expected for such a small corpus squeezed into two axes.

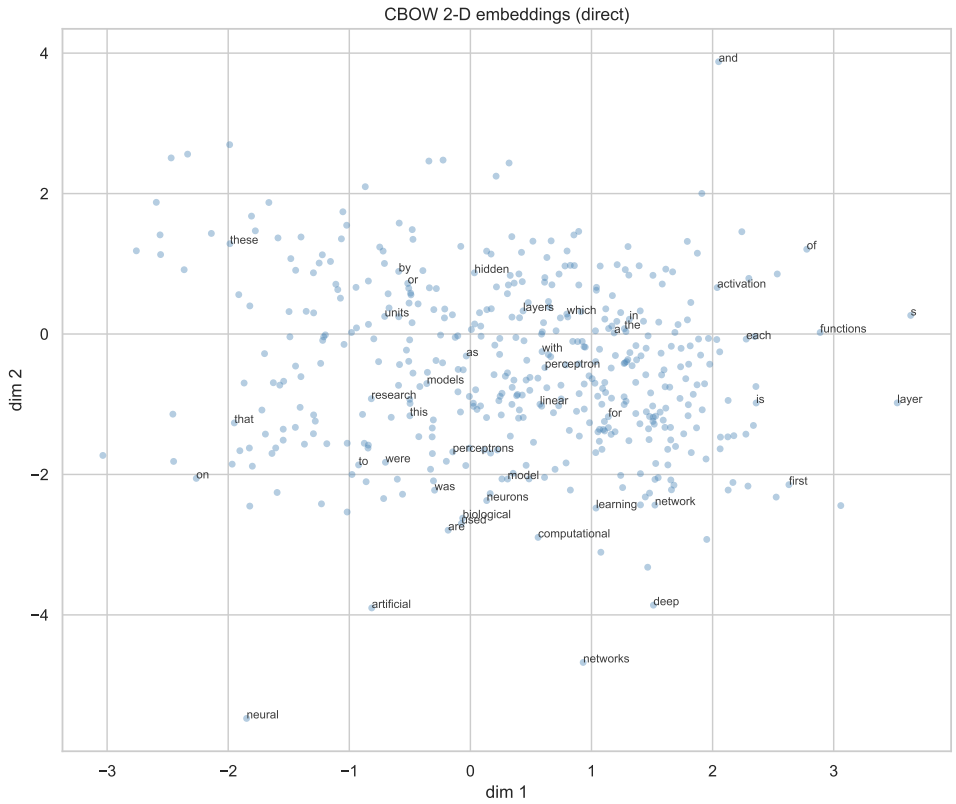


Figure 7: Directly learned 2-D CBOW embeddings (frequent words labelled).

5.2 10-D embedding reduced with PCA

I then learn embeddings of dimension $d = 10$, extract the embedding matrix $E_{10} \in \mathbb{R}^{V \times 10}$, and project it to two dimensions with PCA (Figure 8).

Figure 8: 10-D CBOW embeddings projected to 2-D with PCA.

Comparison. The 10-D model has more room to separate word senses, so after PCA the related-word groups are usually a little cleaner and better spread than in the direct 2-D model, even though the overall layout is similar. The direct 2-D model must pack everything into two axes, which can merge unrelated words, whereas PCA of the richer 10-D space keeps the two most informative directions. Concretely, the 10-D embeddings give much cleaner neighbours: the closest words to *network* become *networks* and *convolutional*, and the closest to *learning* become *algorithm* and *transfer* – associations that are far more sensible than the noisy 2-D neighbours. The 10-D training loss also drops much lower (1.79 vs. 4.62), confirming that ten dimensions capture the co-occurrence structure better before PCA compresses them for display. Both models, however, remain limited by the small corpus.

6 Comprehensive Discussion

Early stopping and dropout vs. the bias–variance trade-off. Both methods aim to reduce variance at the price of a little bias. In our low-noise regression the model hardly overfits, so early stopping never had to fire: it left bias and variance essentially unchanged and simply matched the baseline. Dropout at $p = 0.5$, in contrast, removed so much capacity from the ten-unit network that it added a large amount of bias (underfitting), which is exactly why its test error was far higher. The lesson is that these regularizers only help when there is variance to remove; applied to a model that already fits the signal well, heavy dropout mostly hurts.

Why Adam often converges faster than SGD. Adam rescales each parameter's step by a running estimate of the gradient magnitude, so it makes fast progress even on a poorly conditioned loss surface, and its momentum term smooths the path. SGD with a fixed learning rate must move more cautiously. The trade-off is that Adam's adaptive steps sometimes gener-

alise slightly worse or wobble late in training; on MNIST both optimizers reach a similar final accuracy, but Adam gets there sooner.

Inductive biases of CNNs and LSTMs. Convolutions assume locality and translation invariance: the same small filters detect edges and textures anywhere in the image and the weights are shared, which matches natural images and keeps the parameter count low. LSTMs assume sequential dependence: they read tokens in order and carry a gated memory that preserves long-range context, which matches how sentiment accumulates across a review. Each architecture builds in the right assumption for its data type, and that is why they do well on images and text respectively.

Embedding dimensionality and CBOW. Higher-dimensional embeddings can encode more distinctions, so 10-D vectors reduced by PCA usually separate related words a little better than embeddings learned directly in 2-D. CBOW is simple, fast, and effective at capturing distributional similarity, but it ignores word order inside the window, needs a reasonably large corpus to shine, and gives a single vector per word without context sensitivity.

Practical considerations. The regression and MNIST MLPs train in seconds and are trivial to implement. The CIFAR-10 CNN and IMDB LSTM are heavier: they clearly benefit from a GPU, use most of a 4 GB card at batch 64, and take minutes per run. CNNs parallelise very well, while LSTMs are more sequential and therefore slower per epoch. CBOW is lightweight, but its usefulness grows with the size of the corpus. Overall, matching the model's inductive bias and regularization to the data and the compute budget matters as much as the raw choice of architecture.